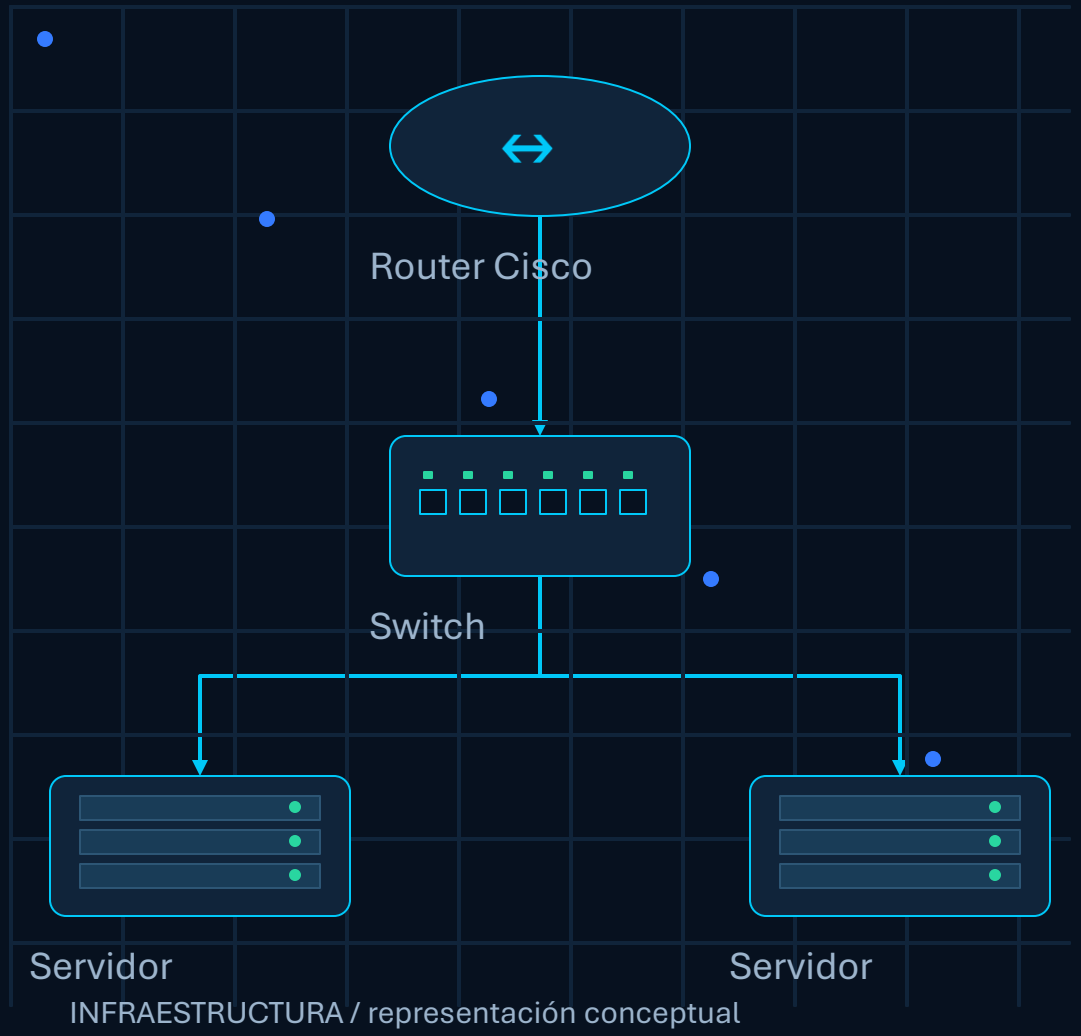
NETAUDIT

Auditoría determinista Cisco IOS / IOS XE

Evidencia trazable y explicación asistida por IA

Axl Urrutia Thomas Henriquez
Maikol Mamani Emilio Cortes

Ingeniería en Conectividad y Redes · Duoc UC
Defensa ante comisión · 1 de diciembre de 2026



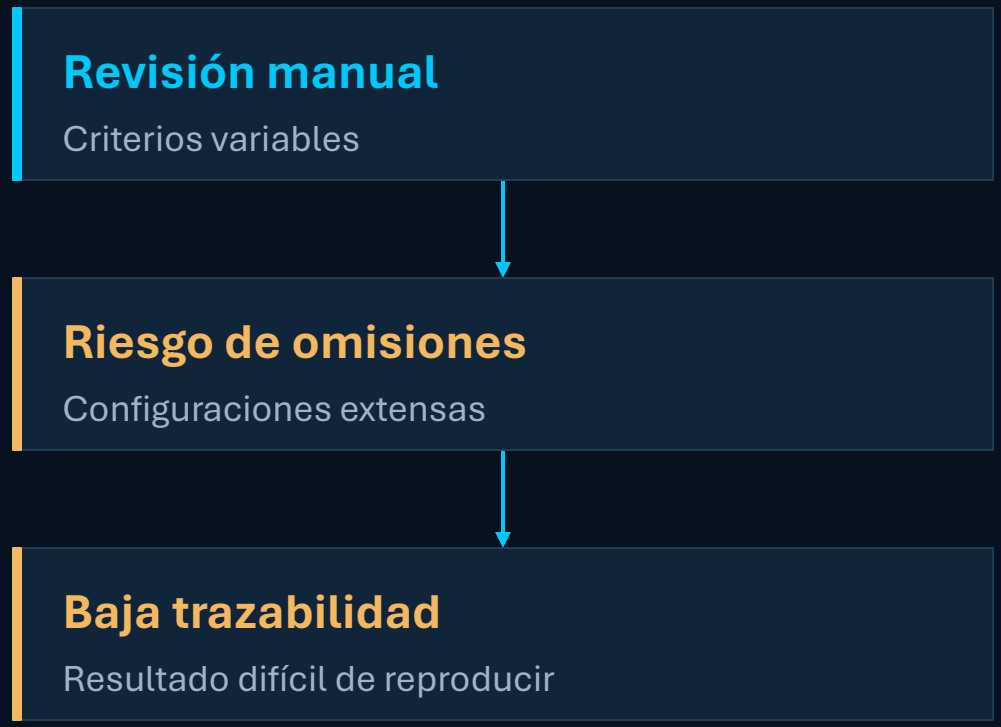
INFRAESTRUCTURA / representación conceptual

De la configuración al riesgo de omisión

02

running-config · fragmento sintético

```
version 16.9
hostname escenario-a
no ip http server
ip ssh version 2
ip finger
service pad
ip source-route
```



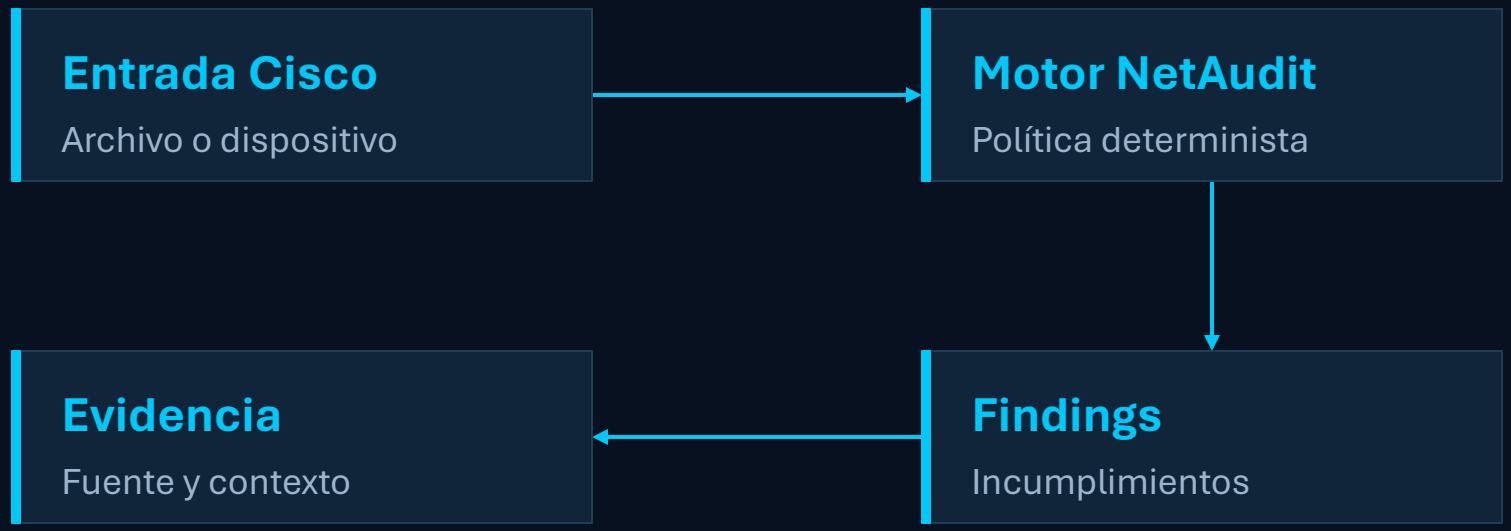
La revisión debe explicar qué se detectó y dónde está la evidencia.

Auditoría convertida en evidencia

24

reglas deterministas

17 configuración
7 operacionales



Recomendaciones para revisión del administrador

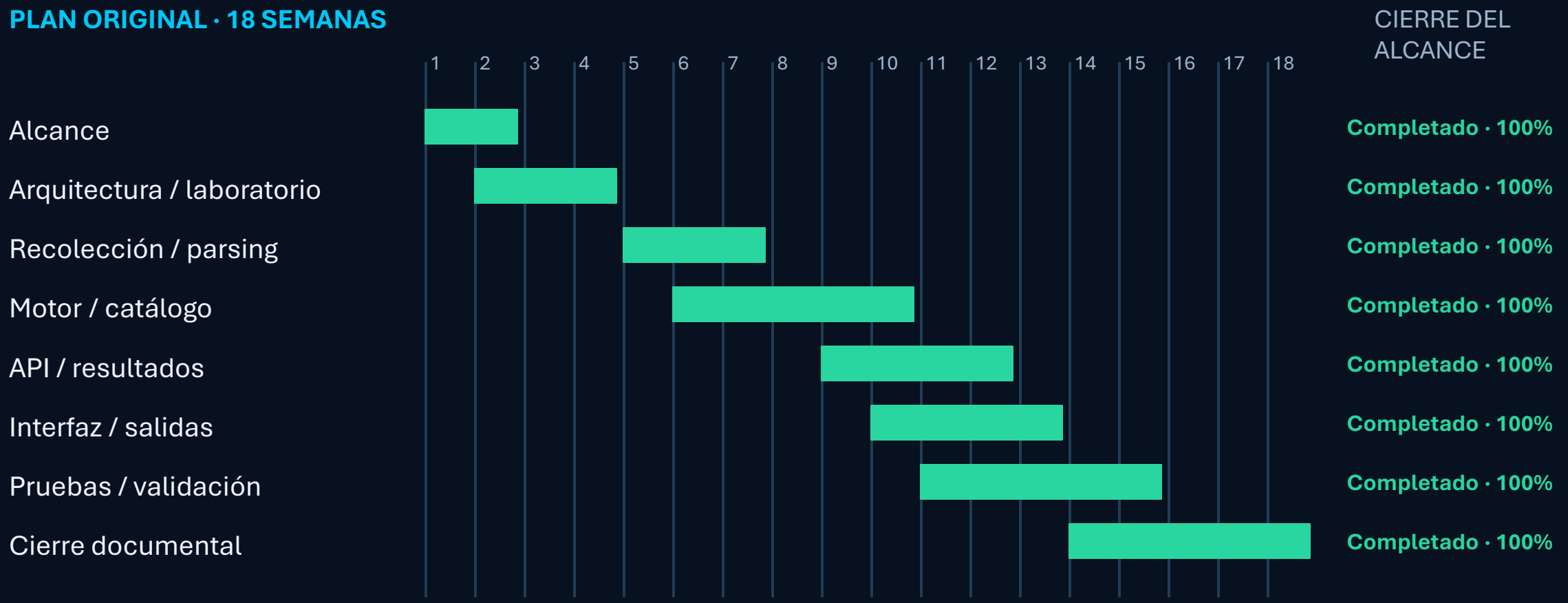
SOLO LECTURA

EVIDENCIA TRAZABLE

IA OPCIONAL

Cierre de la planificación del proyecto

PLAN ORIGINAL · 18 SEMANAS



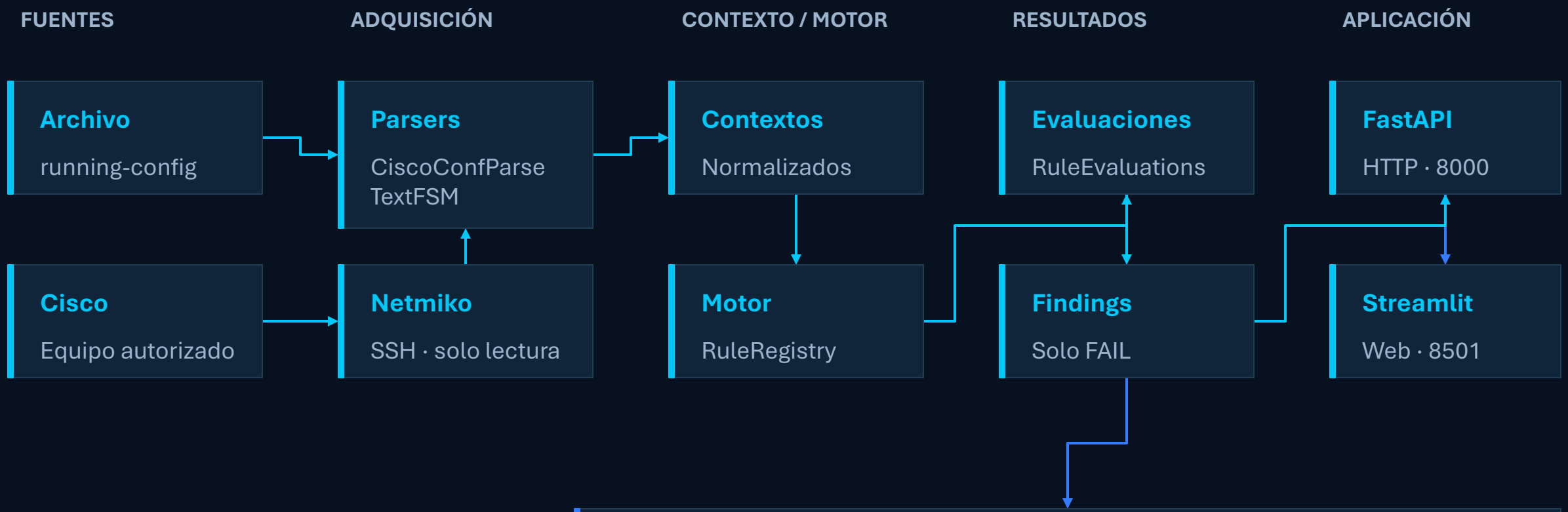
100% — Proyecto finalizado

8 de 8 actividades completadas

Hito final: Entrega de Fase 3 y defensa del proyecto

Una arquitectura con responsabilidades claras

05

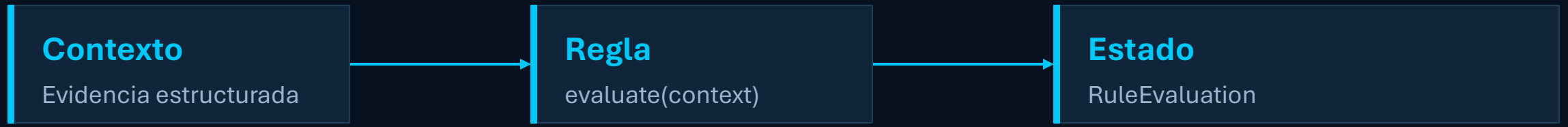


Cian: flujo interno Python
Azul: comunicación HTTP
Cisco → Netmiko: SSH

Sanitización → OmniRoute → Gemini → Validación

Capa explicativa opcional · HTTP después de Findings

El motor conserva cada evaluación



analyzer.py · analyze_context

```
active_registry = registry or get_default_registry()
evaluations = tuple(
    _evaluate_rule(rule, context)
    for rule in active_registry.list_rules(enabled_only=True)
)
findings = findings_from_evaluations(evaluations)
```

- PASS
- FAIL
- NOT_APPLICABLE
- NOT_EVALUATED
- ERROR

Solo FAIL genera Finding

Muestra de código

07

Interfaz Streamlit

apps/streamlit/app.py · 157–159

```python
# Streamlit vuelve a ejecutar esta función al interactuar; session_state conserva el análisis entre ejecuciones.
def main():
    st.set_page_config(page_title='NetAudit · Auditoría de redes', layout='wide', page_icon=':material/hub:')
```

Comunicación con backend

ui/backend.py · 44–46

```python
    # Sube el archivo a FastAPI usando el contrato multipart del endpoint.
    def analyze_file(self, filename: str, content: bytes) -> dict:
        return self._request("POST", "/api/v1/analyses", files={"file": (filename, content, "application/octet-stream")})
```

Análisis determinista

services/analyzer.py · 115–120

```python
# Recorre las reglas habilitadas y devuelve evaluaciones y hallazgos relacionados.
def analyze_context(context, *, registry: RuleRegistry | None = None) -> AnalysisResult:
    active_registry = registry or get_default_registry()
    evaluations = tuple(
        _evaluate_rule(rule, context)
        for rule in active_registry.list_rules(enabled_only=True)
```

Integración con IA

ai/client.py · 62–65

```python
            # El destino es el gateway configurado. El límite de bytes se aplica mientras se lee la respuesta.
            with client.stream("POST", self.settings.base_url + "/chat/completions",
                               headers=self._headers(), json={
                                   "model": self.settings.model, "messages": messages,
```

Capturas reales de VS Code · Comentarios explicativos del código versionado

La IA recibe resultados ya determinados

01 MOTOR DETERMINISTA

services/rule_results.py · filtro de findings_from_evaluations

```
    for evaluation in evaluations
    if evaluation.status is RuleStatus.FAIL
```

02 CAPA IA OPCIONAL

Findings → Sanitización → OmniRoute → Gemini → Validación

ai/client.py · _complete_once · extracto HTTP

```
with client.stream("POST", self.settings.base_url + "/chat/completions",
                   headers=self._headers(), json={
                       "model": self.settings.model, "messages": messages,
                       "temperature": 0, "max_tokens": 4000, "stream": False,
                   }) as response:
```

Gemini explica; no determina. Validación Pydantic e identificadores permitidos.

Especialización y automatización

09

	ANSIBLE Automatización general	NETAUDIT Auditoría Cisco especializada
Recolección	ios_command	Netmiko / ocho consultas
Automatización	Operaciones de infraestructura	Análisis de auditoría
Evaluación	Lógica de auditoría adicional	24 reglas predefinidas
Evidencia	Según playbooks y salidas	Contrato de evaluación y Finding
Explicación	Según integración elegida	IA opcional y validada
Configuración	ios_config puede aplicar cambios	No aplica configuraciones

Pueden complementarse. Ansible también permite construir auditorías.

Dos entradas, un criterio de auditoría

ARCHIVO



DISPOSITIVO

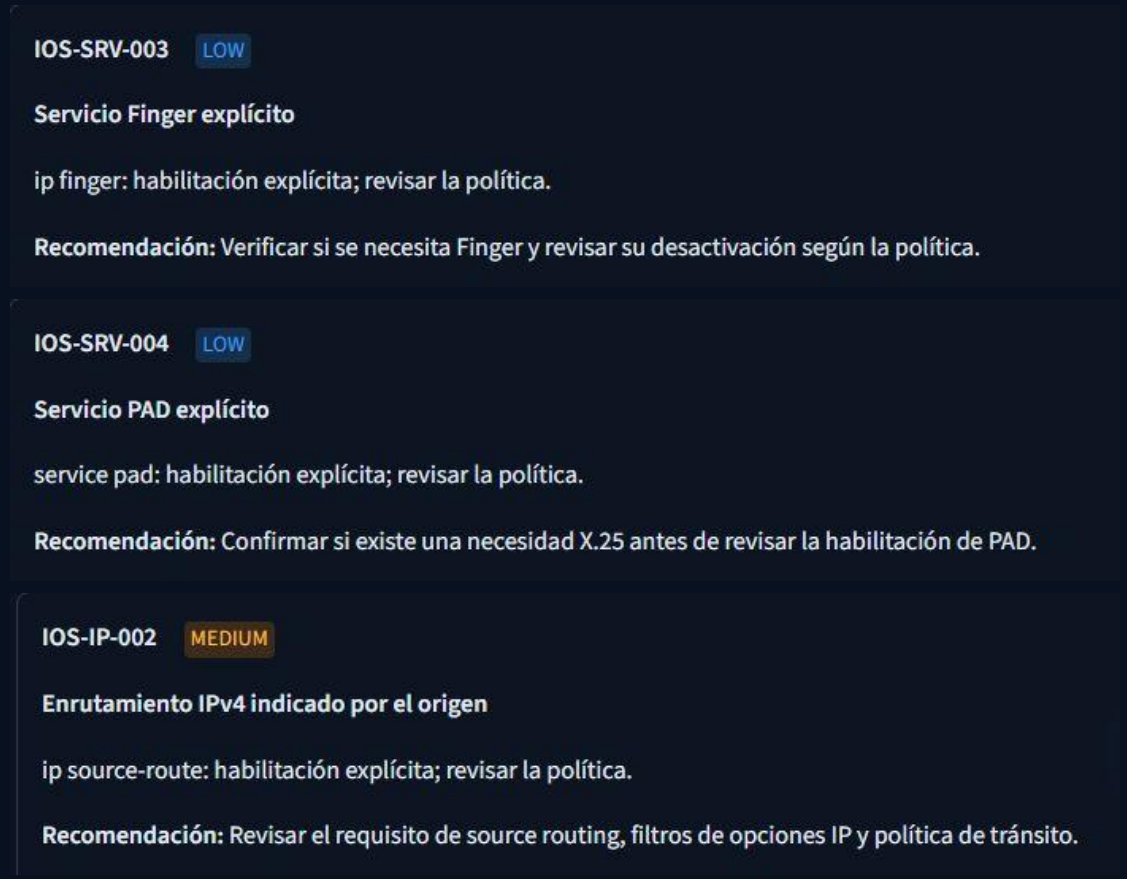


Archivo: sin conexión SSH. Dispositivo: ocho consultas autorizadas de lectura.

Las pruebas offline no equivalen a una conexión real.

Evidencia real del prototipo

17 evaluaciones · 3 Findings · archivo sintético



IOS-SRV-003 LOW

Servicio Finger explícito

ip finger: habilitación explícita; revisar la política.

Recomendación: Verificar si se necesita Finger y revisar su desactivación según la política.

IOS-SRV-004 LOW

Servicio PAD explícito

service pad: habilitación explícita; revisar la política.

Recomendación: Confirmar si existe una necesidad X.25 antes de revisar la habilitación de PAD.

IOS-IP-002 MEDIUM

Enrutamiento IPv4 indicado por el origen

ip source-route: habilitación explícita; revisar la política.

Recomendación: Revisar el requisito de source routing, filtros de opciones IP y política de tránsito.

IOS-SRV-003 · LOW

`ip finger`

Evidencia · línea 37

IOS-SRV-004 · LOW

`service pad`

Evidencia · línea 38

IOS-IP-002 · MEDIUM

`ip source-route`

Evidencia · línea 39

Capturas reales · 10/11/2026 · demo_servicios.cfg · anotaciones de evidencia a la derecha

Validación del alcance implementado

852
Pruebas aprobadas

72/72
Comparaciones offline

24
Reglas integrales

0
Fallos de la suite

8 consultas de solo lectura

IA separada del motor

Errores del proveedor controlados

Corte comprobado: 11/11/2026 · 1 warning de Starlette/AnyIO

Estas métricas no acreditan cobertura total ni validación integral en producción.

NETAUDIT

Las reglas detectan.

La evidencia demuestra.

La IA explica.



Auditoría reproducible · Solo lectura · Alcance y límites documentados